COMP3121 Notes

Ronnie Low

Questions to-be-answered

1) for big theta, if big oh and big omega are different then do we have a big theta?
2) Proof by invareints review

What is an algorithm?
Algorithms are a set of precisely defined steps are executed based off mechanical steps - there is no intelligence involved, just follow the steps given.

Types of algorithms
1. Sequential deterministic algorithms:
- Only one step can be executed at a time
- No element of randomness

2. Parallel algorithms:
- Multiple steps can be ran at a time
- Probably not covered in 3121

3. Randomised algorithms:
- Utilises random values
- Used in quantum computing
- Probably not covered in 3121

Why prove algorithms?
Sometimes, its not obvious if an algorithm does the things its advertised through common sense:
- Will your algorithm terminate in a timely manner?
- Can you prove that the algorithm won't infinitely loop?
- Is the time complexity as advertised?

Proofs are needed for these cases, most of the time they're the **only** way to know if the algorithm does its job.

Stable matching example
There are *n* hospitals and *n* doctors. Each hospital wants to hire exactly one doctor. Each doctor and hospital submit a list of preferences ranking which hospital they want to go to and which doctor they want.

How do we design an algorithm where there is no hospital doctor pair (*h, d*) that are dissatisfied with their result (stable):

$$\text{for pair } p = (h, d') \text{ \& } p' = (h', d)$$

- h prefers d over d'
- h' prefers d' ovr d

This implies that both hospitals and doctors have a preferred swap to make (unstable match)

Stable matching algorithm
While there is a free hospital, pick a free hospital h and offer a job to the highest ranking doctor d on its preference list.

If d doesn't have a job yet, d always accepts the job.
else if d is already in a pair $(h'd)$:
- if h is higher ranked on the preference list for d, a new pair (h, d) is created and the previous pair (h', d) is deleted.
- if h is ranked lower, then the offer is ignored.

How many ways to match hospital-doctors?
$n!$, first doctor has n choices, next doctor has $n - 1$ choices, so on and so forth.
Will a stable match always exist?
Yes, but it is not obvious - we need to use a proof:
note that during the while loop:
- for a doctor, the hospital they go to will always be higher on their ranks.
- a hospital will be paired with doctors of decreasing ranks.

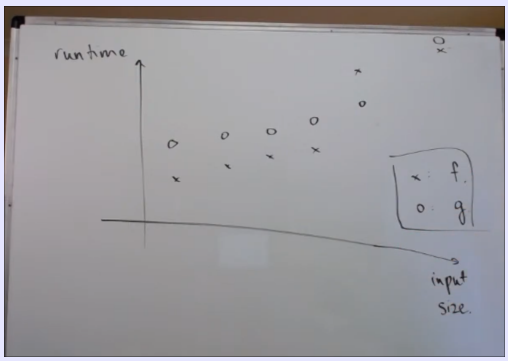
If we assume that the ranking is not stable: $p = (h, d)$ and $p' = (h', d')$ such that h prefers d' and h' prefers d.

Since h prefers d' to d, then it must have made an offer to d' before offering to d, but since h is paired with d, then d' would need to have rejected h because tehy were already at a hospital they prefer to h. This proves that its impossible to have d' prefer h' if h is ranked higher.

Rates of growth (Recap)
There are statements like:
- *Heap sort is faster than bubble sort.*
- *Linear search is slower than binary search.*

We want our language to be more precise; when is it faster, will it be not the case?

If we were to compare two algorithms, issues may occur!
- Outliers! We can see in the graph above that although g is generally slower, the fifth point has an outlier where f is slower. How do we describe the changes/differences?
- Implementation differences: what if f' is slower than both g and f, but f' has a different implementation?



We mostly perfer to talk in terms of long term behaviour (asymtopics)

For instance, if the size of an input doubles, the function could double, triple, or even quadruple!

Big O Notation

We say that $f(n) = O(g(n))$ if:

There are positive constants C and N such that $0 \leq f(n) \leq C \times g(n)$ for all $n \geq N$.

What does that acutally mean? lets take a look at some examples.

Is $f(n) = O(g(n))$ for:
$f(n) = n + 5$, $g(n) = n^2 - 1$

For this to hold true, there needs to be a value n and C such that every single value of $f(n) \leq g(n)$. **Is this true?**

Well, given a sufficiently large n, yes! Lets prove it:
Lets assign $C = 1$

$$f(n) = n + 5 < g(n) = n^2 - 1 \tag{1}$$
$$0 < n^2 - n - 6 \tag{2}$$
$$0 < (n-3)(n+2) \tag{3}$$
$$\therefore n \text{ is true for } n > 3 \text{ or } n < -2 \tag{4}$$

this implies that for values $C = 1$ and every value $n > 3$, this equation holds true!
In more human terms, we **need** to have a value C that is multiplied with $g(n)$, and there **needs** to exist a value n such that every value greater than n will satisfy

$$0 \leq f(n) \leq C \times g(n).$$

In doing so, we can say that the for $f(n)$ (which is the function of input size vs time) has a time complexity of $O(g(n))$ (keep in mind $g(n)$ will just be an equation that $f(n)$ will never exceed.)

Keep in mind that big O tends to overestimate the complexity (by virute of upper bounding at an asymtopic point).

Is big-oh the be all-end all?

We know that big-oh demonstrates the worst case time complexities. But theres alot of data we're missing out on! What about best case? There are algorithms out there that can run faster than other algos even if they are bounded by the same big-oh time complexity! Think about running bubble sort on a sorted array with the first and last items swapped vs selection sort. This is where big theta and omega comes in.

Big Omega

Big omega is the inverse of big-oh, it yields the asymtopic lower bound of a function. In plain words, $\Omega(f(n))$ is a metric that given a long enough running time, $f(n)$ will always be at least $\Omega(f(n))$. Another way to think about it is that given a time complexity $f(n)$, it will never be less than $\Omega(f(n))$ so plotting it on a graph will show that $\Omega(f(n))$ is bounded by $f(n)$.

Big theta

given that we know $O(f(n))$ and $\Omega(f(n))$, we can combine those two graphs (where the space between big-oh and big omega represents big theta!). This means that the growth rate is the same in both best and worst case senarios.

sum and product property

for all time complexities, we want to know how summing differnet time complexities and the product of them chagnes the total time complexity!

for the sum of time complexities, we take the larger value:

$$O(a) > O(b), O(a) + O(b) = O(a)$$

. As an example, this makes sense!

$$O(n) + O(n^2) = O(n^2)$$

for the product of time complexities, the product of two time complexities will result in the product of the growth rates:

$$O(a) \times O(b) = O(ab)$$

.

Heaps, Frequency tables and Hash tables

To be honest, I don't want to explain this so this will be a challenge left to the reader.

Propositional Logic

- Propositions are true/false statements. We can run operations on them!
- $\neg P$ is a negation of P, i.e. !P.
- $P \vee Q$ is P OR Q, P $\|$ Q
- $P \wedge Q$ is P AND Q, P && Q
- $P \Rightarrow Q$ is P implies Q, i.e. if P is true, then Q

is true.

- $P \iff Q$ is P if and only if Q which means that for P to true, Q also has to be true, and for Q to be true, then P also has to be true. People have explained iff as $P \Rightarrow Q, Q \Rightarrow P$, but I like to think of it as "they are either both true or they are either both false"

Types of proofs
In 3121, we need to prove algorithms. There are a couple techniques we can employ to do so:

- Proof by induction
- Proof by invarients
- Proof by contradiction

Proof by induction
Proof by induction is pretty fun. Generally, induction is used to prove a sequence of things (that are ordered). If we prove that given any position, the next one holds a certain property and the first item also holds that property, then we can extrapolate that the entire set holds the property. For instance, for an infinitely long set of stairs that are green, if we are given the property that at any green step, the next step will be green.

There exists another kind of induction, strong induction. Normal indcution assumes something at a point: at k, the $k+1$th step is green. However for strong induction, we assume it is true from the base case to k, i.e. $1, 2, ..., k$th steps are all green.

To prove for induction, we have to prove three things:

1) the base case is correct
2) assume it is true for a k (or up to k if we are doing strong induction)
3) prove it is true for the $k+1$th item

An induction proof is not complete without these three properties.

Proof by invarients
Proof by invarients is pretty similar to induciton: We wnat to prove that a property P holds true over an algorithm, and that if P is true at the start, and if we can prove that at any step we can preserve P, then P holds true throughout.

Proof by contradiction
We can prove that something is true by thinking about its negation. If we assume that $\neg P$ is true, and that we somehow prove that this is false, then we can prove that P is true. This feels flimsy but it holds true.

For any statement P, it either has to be true or false. Negating that statement P means that $\neq P$ is the opposite of P. As such, if $\neq P$ is false, then P is true.

Algorithm Reeasoning: Mergesort

- For an algorithm, we need to prove correctness and efficiency
- as such, we need to prove that it gives the right answer and jusitfy the time complexity

Lets take mergesort for example: We would need to prove that not only mergesort "sorts" correctly, but also it runs in $O(nlogn)$ time for a proof to be complete.

Mergesort takes an array $A[l..r]$, and splits it in half every time. If $l = r$, then it simply returns (since there is just one item and theres nothing left to sort).

- Otherwise, you get the midpoint: $(l+r)/2$.
- Apply the algorithm recursively to $A[l, m]$ and $A[m+1, r]$
- merge the subarrays $A[l..m]$ and $A[m+1..r]$, into a sorted subarray $A[l..r]$.

We need a merge operation to successfully merge it though:

- Create a buffer B of size $r - l + 1$.
-